def analyze_sentence(sentence): words = sentence.split() word_count = len(words)

```
    char_count = len(sentence.replace(" ", ""))

    unique_words = len(set(word.lower() for word in words))

    freq = {}
    for word in words:
        word_lower = word.lower()
        freq[word_lower] = freq.get(word_lower, 0) + 1

    most_frequent_word = max(freq, key=freq.get)

    return {
        "word_count": word_count,
        "char_count": char_count,
        "unique_words": unique_words,
        "most_frequent_word": most_frequent_word
    }
```

result = analyze_sentence('The quick brown fox jumps over the lazy dog the fox') print(result)

In [4]:
```
data = [("Alice", 92, "A"), ("Bob", 78, "B+"), ("Charlie", 55,
"C"), ("Diana", 100, "A+")]


def format_report(data):
    print(f"{'Name':<15}{'Score':>6}{'Grade':^8}")
    print("-" * 29)

    for name, score, grade in data:
        print(f"{name:<15}{score:>6}{grade:^8}")


data = [
    ("Alice", 92, "A"),
    ("Bob", 78, "B+"),
    ("Charlie", 55, "C"),
    ("Diana", 100, "A+")
]

format_report(data)
```

```
Name            Score Grade
-----------------------------
Alice              92   A
Bob                78   B+
Charlie            55   C
Diana             100   A+
```

In [ ]:
```
section B Data Structures
```

In [ ]:
```
4. Write a function list_ops(nums) that:
a) Removes duplicates while preserving the original order.
b) Returns a new list with only elements at even indices, squared.
c) Flattens the nested list into a single list using list comprehension.
```

In [ ]:
```
nums = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
nested = [[1, 2, 3], [4, 5], [6, 7, 8, 9]]
```

In [3]:
```
def list_ops(nums, nested):
    # a) Remove duplicates while preserving order
    unique = []
    for x in nums:
        if x not in unique:
            unique.append(x)

    # b) Elements at even indices, squared
    squared_even_indices = [nums[i] ** 2 for i in range(0, len(nums), 2)]

    # c) Flatten nested list
    flattened = [item for sublist in nested for item in sublist]

    return unique, squared_even_indices, flattened


nums = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
nested = [[1, 2, 3], [4, 5], [6, 7, 8, 9]]

result = list_ops(nums, nested)

print("Without duplicates:", result[0])
print("Even-index elements squared:", result[1])
print("Flattened list:", result[2])
```

```
Without duplicates: [3, 1, 4, 5, 9, 2, 6]
Even-index elements squared: [9, 16, 25, 4, 25, 25]
Flattened list: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [ ]:
```
5.Given the tuple below:
employee = ('E042', 'Riya Sharma', 'Data Science', 72000, 'Pune')
a) Unpack it into meaningful variable names.
b) Use * unpacking to separate the first element, the last, and everything in betwe
c) Write a function `update_salary(emp_tuple, new_salary)` that returns a new tuple
```

In [5]:
```
# Given tuple
employee = ('E042', 'Riya Sharma', 'Data Science', 72000, 'Pune')

# a) Unpack into meaningful variable names
emp_id, name, department, salary, city = employee
print("a) Unpacked values:")
print("Employee ID:", emp_id)
print("Name:", name)
print("Department:", department)
print("Salary:", salary)
print("City:", city)
```

```
# b) Use * unpacking to separate first, last, and everything in between
first, *middle, last = employee
print("\nb) Using * unpacking:")
print("First element:", first)
print("Middle elements:", middle)
print("Last element:", last)

# c) Function to update salary and return new tuple
def update_salary(emp_tuple, new_salary):
    emp_id, name, department, salary, city = emp_tuple
    return (emp_id, name, department, new_salary, city)

# Test the function
updated_employee = update_salary(employee, 85000)
print("\nc) Updated tuple:", updated_employee)
```

```
a) Unpacked values:
Employee ID: E042
Name: Riya Sharma
Department: Data Science
Salary: 72000
City: Pune

b) Using * unpacking:
First element: E042
Middle elements: ['Riya Sharma', 'Data Science', 72000]
Last element: Pune

c) Updated tuple: ('E042', 'Riya Sharma', 'Data Science', 85000, 'Pune')
```

In [ ]:
```
6.Using the sets below, find:
batch_a = {'Aarav', 'Priya', 'Karan', 'Meera', 'Sahil'}
batch_b = {'Karan', 'Meera', 'Tanvi', 'Rohan'}
batch_c = {'Priya', 'Aarav'}
a) Students present in both batches.
b) Students in batch_a but not in batch_b.
c) All unique students across both batches.
d) Whether batch_c is a subset of batch_a.
```

In [6]:
```
batch_a = {'Aarav', 'Priya', 'Karan', 'Meera', 'Sahil'}
batch_b = {'Karan', 'Meera', 'Tanvi', 'Rohan'}
batch_c = {'Priya', 'Aarav'}

# a) Students present in both batches - intersection
both_batches = batch_a & batch_b
print("a) Present in both batch_a and batch_b:", both_batches)

# b) Students in batch_a but not in batch_b - difference
only_in_a = batch_a - batch_b
print("b) In batch_a but not in batch_b:", only_in_a)

# c) All unique students across both batches - union
all_unique = batch_a | batch_b
print("c) All unique students in batch_a and batch_b:", all_unique)

# d) Whether batch_c is a subset of batch_a
```

```
is_subset = batch_c.issubset(batch_a)
print("d) Is batch_c a subset of batch_a?", is_subset)
```

```
a) Present in both batch_a and batch_b: {'Karan', 'Meera'}
b) In batch_a but not in batch_b: {'Priya', 'Aarav', 'Sahil'}
c) All unique students in batch_a and batch_b: {'Karan', 'Aarav', 'Sahil', 'Meera',
'Tanvi', 'Rohan', 'Priya'}
d) Is batch_c a subset of batch_a? True
```

In [ ]:
```
7. Write a function inventory_manager(records) that takes a list of tuples (product
records = [
    ('Laptop', 'Electronics', 10, 55000),
    ('Mouse', 'Electronics', 50, 499),
    ('Desk', 'Furniture', 8, 12000),
    ('Chair', 'Furniture', 15, 4500),
    ('Pen', 'Stationery', 200, 20)
```

In [7]:
```
def inventory_manager(records):
    result = {}

    for product, category, quantity, price in records:
        total_value = quantity * price

        if category not in result:
            result[category] = {
                'total_items': 0,
                'total_value': 0,
                'products': []
            }

        result[category]['total_items'] += quantity
        result[category]['total_value'] += total_value
        result[category]['products'].append(product)

    return result

# Test data
records = [
    ('Laptop', 'Electronics', 10, 55000),
    ('Mouse', 'Electronics', 50, 499),
    ('Desk', 'Furniture', 8, 12000),
    ('Chair', 'Furniture', 15, 4500),
    ('Pen', 'Stationery', 200, 20)
]

# Function call + print
output = inventory_manager(records)
print(output)
```

```
{'Electronics': {'total_items': 60, 'total_value': 574950, 'products': ['Laptop', 'M
ouse']}, 'Furniture': {'total_items': 23, 'total_value': 163500, 'products': ['Des
k', 'Chair']}, 'Stationery': {'total_items': 200, 'total_value': 4000, 'products':
['Pen']}}
```

In [ ]:
```
Section C – Functions and Modules
```

In [ ]:
```
8. Implement the following:
a) A function power_all(_args, exp=2) that returns a list of each number raised to
b) A function profile(_*kwargs) that prints each key-value pair as Key : Value.
c) Use a lambda inside sorted() to sort the list of dicts below by cgpa in descendi
students = [
    {'name': 'Arjun', 'cgpa': 8.4},
    {'name': 'Dev', 'cgpa': 7.8},
    {'name': 'Sanya', 'cgpa': 9.1}
]
```

In [8]:
```
# a) Function power_all(*args, exp=2)
def power_all(*args, exp=2):
    return [x ** exp for x in args]

# b) Function profile(**kwargs)
def profile(**kwargs):
    for key, value in kwargs.items():
        print(f"{key} : {value}")

# c) Sort list of dicts by cgpa in descending order using lambda
students = [
    {'name': 'Arjun', 'cgpa': 8.4},
    {'name': 'Dev', 'cgpa': 7.8},
    {'name': 'Sanya', 'cgpa': 9.1}
]

sorted_students = sorted(students, key=lambda x: x['cgpa'], reverse=True)

# Testing everything
print("a) power_all(2, 3, 4):", power_all(2, 3, 4))
print("a) power_all(2, 3, 4, exp=3):", power_all(2, 3, 4, exp=3))

print("\nb) profile output:")
profile(name="Riya", age=20, city="Pune")

print("\nc) Sorted students by cgpa descending:")
for s in sorted_students:
    print(s)
```

```
a) power_all(2, 3, 4): [4, 9, 16]
a) power_all(2, 3, 4, exp=3): [8, 27, 64]

b) profile output:
name : Riya
age : 20
city : Pune

c) Sorted students by cgpa descending:
{'name': 'Sanya', 'cgpa': 9.1}
{'name': 'Arjun', 'cgpa': 8.4}
{'name': 'Dev', 'cgpa': 7.8}
```

In [ ]:
```
10. Complete the following:
a) Write a closure make_multiplier(n) that returns a function multiplying its input
```

```
b) Write a decorator @timer that prints the execution time of any function it wraps

c) Write a decorator @validate_positive that raises a ValueError if any argument is
```

In [9]:
```python
import time
import math
from functools import wraps

def make_multiplier(n):
    def multiplier(x):
        return x * n
    return multiplier

triple = make_multiplier(3)
quintuple = make_multiplier(5)

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end - start:.6f} seconds")
        return result
    return wrapper

@timer
def slow_sum(n):
    return sum(i**2 for i in range(1, n+1))

def validate_positive(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        for arg in args:
            if arg <= 0:
                raise ValueError("All arguments must be positive")
        return func(*args, **kwargs)
    return wrapper

@validate_positive
def safe_sqrt(x):
    return math.sqrt(x)

print(triple(10))
print(quintuple(7))
slow_sum(1000)
print(safe_sqrt(16))
```

```
30
35
slow_sum took 0.000159 seconds
4.0
```

In [ ]:
```
2. Given the string below, do the following and print each result:

text = "A man a plan a canal Panama"
```

```
a) Reverse the string without using slicing.
b) Check if it is a palindrome, ignoring case and spaces.
c) Replace every vowel with *.
d) Count how many times each vowel appears.
```

In [10]:
```python
text = "A man a plan a canal Panama"

# a) Reverse the string without using slicing
reversed_text = ''.join(reversed(text))
print("a) Reversed:", reversed_text)

# b) Check if it is a palindrome, ignoring case and spaces
clean_text = ''.join(text.lower().split())
is_palindrome = clean_text == clean_text[::-1]
print("b) Is palindrome:", is_palindrome)

# c) Replace every vowel with *
vowels = "aeiouAEIOU"
replaced_text = ''.join('*' if ch in vowels else ch for ch in text)
print("c) Vowels replaced:", replaced_text)

# d) Count how many times each vowel appears
vowel_count = {v: 0 for v in "aeiou"}
for ch in text.lower():
    if ch in vowel_count:
        vowel_count[ch] += 1
print("d) Vowel counts:", vowel_count)
```

```
a) Reversed: amanaP lanac a nalp a nam A
b) Is palindrome: True
c) Vowels replaced: * m*n * pl*n * c*n*l P*n*m*
d) Vowel counts: {'a': 10, 'e': 0, 'i': 0, 'o': 0, 'u': 0}
```

In [ ]: